Opus.Prism: Comprehensive Technical Documentation

Sip N' Hack Project Submission

March 9, 2026

Abstract

Opus.Prism is an agentic multimodal video personalization compiler and automated QA testing suite. It addresses core bottlenecks in AI video generation: lack of audience personalization, hallucinated on-screen text, and the absence of an automated critic-in-the-loop. Built by a former YouTube creator, this system ensures content is strategically tailored and visually verified before ever reaching an audience.

Important Project Links:

- **Video Demo:** Watch on Google Drive
- **GitHub Repository:** github.com/chetas1208/opus.Prism
- **Creator's Background:** YouTube Channel

Contents

1 Introduction & Creator Background **2**

2 Architecture: Hugging Face Open-Weight Integration **2**

3 System Flow & UI Interfaces **3**

3.1 PersonaCut Project Creator (`/personacut`) 4

3.2 Variant Results & The First Critic 4

3.3 TextGuard QA: The Second Critic 5

4 API Configuration & Integrations **6**

5 Conclusion **7**

1 Introduction & Creator Background

As a former YouTube creator (view my channel here), I deeply relate to the core mission of Opus Clip. Producing video content is grueling. You don't just create one video; you must tailor it for different formats, durations, and platforms. Current generative AI pipelines fall short—they generate one generic video from a prompt and ship it blindly, often with garbled text overlays that ruin the professional polish.

Opus.Prism directly solves these workflow bottlenecks by merging three Sip N' Hack problem statements into one cohesive system:

1. **Problem 1 (Personalization):** Handled by **PersonaCut**, which compiles a single source script into multiple audience-specific variant packs (differing in tone, duration, platform, and pacing).
2. **Problem 5 (Critic in the Loop):** Handled by pre-generation Scorecards and post-generation VLM inspectors.
3. **Problem 3 (Accurate Text / No Gibberish):** Handled by **TextGuard QA**, extracting frames to verify and patch broken visual text automatically.

2 Architecture: Hugging Face Open-Weight Integration

Opus.Prism has completely migrated away from proprietary APIs. The entire intelligence layer runs locally via Hugging Face's open-weight models using the inference API (or `vLLM`/`LMDeploy`):

- **Variant Generator & PersonaCut Agents:** `Qwen/Qwen2.5-72B-Instruct` (The strongest available open-weight model for complex JSON reasoning and agentic workflows).
- **TextGuard QA Verdicts (Multimodal VLM):** `Qwen/Qwen2.5-VL-7B-Instruct` (Used specifically to inspect failed frames and deduce if OCR failures are due to styling or hallucinated gibberish).
- **OPBot / Light Logic:** `Qwen/Qwen2.5-7B-Instruct` (For the fast, context-aware floating UI chatbot).

Note: Extracted frames are scanned using ***`EasyOCR`*** *before the VLM is deployed as a fallback inspector.*

Opus.Prism Architecture Schematic

```
+-------------------------------------------------------------------------------+
| FRONTEND APP (Next.js / Tailwind)                                             |
|                                                                               |
|  [ /personacut ] ----> [ /results ] ----> [ /qa ]      <-- (OPBot Context Aware) |
+---------|-------------------|----------------|-----------------------A-----------+
          |                   |                |                       |
```

```
          | (2. Trigger       |                | (9. Upload MP4)       | (Chat / TTS)
          |  Pipeline)        |                v                       |
+---------|-------------------|----------------|-----------------------|-----------+
|         v                   |                |     [ Qwen-7B / ElevenLabs ]      |
|  +--------------------+     |                v                                   |
|  | PERSONACUT BACKEND |     |        +--------------+                            |
|  |  (Agent Pipeline)  |     |        | TEXTGUARD QA |                            |
|  +--------------------+     |        +--------------+                            |
|    |                        |          |                                         |
|    |- Agent 1: Extractor    |          |- Frame Extractor (ffmpeg)               |
|    |    (Story Facts)       |          |      |                                  |
|    |                        |          |      v                                  |
|    |- Agent 2: Strategist   |          |- OCR Engine (Qwen2.5-VL)                |
|    |    (Demographics)      |          |      |                                  |
|    |                        |          |      v                                  |
|    |- Agent 3: Writer       |          |--[ Matcher ]                            |
|    |    (Script/Captions)   |          |      | \                                |
|    |                        |          |      |  \ (Match Fails)                 |
|    |                        |          |      |   \                              |
|    |- Agent 4: Planner      |          |      |    v                             |
|    |    (Visual Prompts)    |          |      |  [ VLM Inspector ]               |
|    |                        |          |      |    (Qwen2.5-VL)                  |
|    |- Agent 5: Critic       |          |      |          |                       |
|         (Scorecard)         |          |      |          v                       |
|         /        \          |          |      |  [ Video Patcher ]               |
|  (Pass)/          \(Fail)   |          |      |          \                       |
|       v            \--> (Retry via  |          |      |          v
| [ Variant Packs ]       Agent 2)    |          v          v                      |
|       |                     |          +-------------------------+               |
|       |-------------------- |          | Output: QA Report JSON  |               |
+-------|---------------------|----------|         & Patched MP4   |               |
        |      (3. Returns)   ^          +-------------------------+               |
        v                     |                     |                               |
  (4. Export JSON / Prompt)   |                     v                               |
        |                     |            (10. Feedback / Report Displayed)        |
        |                     |                     |                               |
        v                     |                     v                               |
  [ External Video Generator ]|               (End of Workflow)                     |
    (Runway, Sora, etc.)      |                                                     |
        |                     |                                                     |
        | (5. Render MP4)     |                                                     |
        +---------------------+                                                     |
```

3 System Flow & UI Interfaces

The user interface (built on Next.js) is split into primary workflow phases.

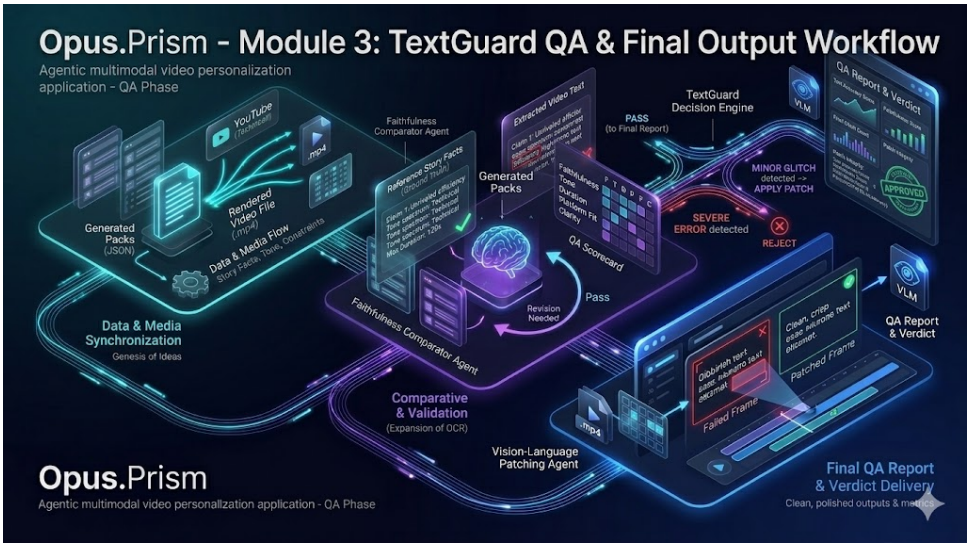

Figure 1: The modular interface of Opus.Prism handling compilation and QA.

3.1 PersonaCut Project Creator (/personacut)

This interface (see Figure 1) asks for the "Master Idea". Rather than generating a video immediately, the system acts as a multi-stage compiler leveraging the `Qwen2.5-72B-Instruct` pipeline.

The 5-Agent Pipeline When a user submits a script alongside multiple target audience profiles (e.g., TikTok Casual vs. LinkedIn Professional), Opus.Prism spins up a localized agent framework:

1. **Story Extractor:** Scans the raw input to lock down "Story Facts." It ensures immutable data like brand names, numerical claims, and core messages cannot be hallucinated away by downstream agents.
2. **Audience Strategist:** Parses the `TargetSpec` JSON. It dictates the pacing, visual density, and compression level required for the specific platform.
3. **Script Writer:** Drafts the tailored voiceover and the exact, time-synced captions.
4. **Shot Planner:** Injects generative visual prompts based on the script timeline, setting up precise scene transitions.

3.2 Variant Results & The First Critic

As shown in Figure 2, the results dashboard displays the generated multimedia JSON plans.

Agent 5: Pre-Generation Scorecard Crucially, Opus.Prism utilizes a **Critic-in-the-Loop** natively. Before a user exports the prompt to an external renderer, a fifth Critic Agent grades the variant. It evaluates the JSON payload against the original Story Facts, scoring it out of 50 points on: Faithfulness, Tone Match, Duration Fit, Platform Fit, and Clarity. If a variant scores below an acceptable threshold, the pipeline automatically triggers a revision loop, ensuring only high-quality prompts reach the final API generation step.

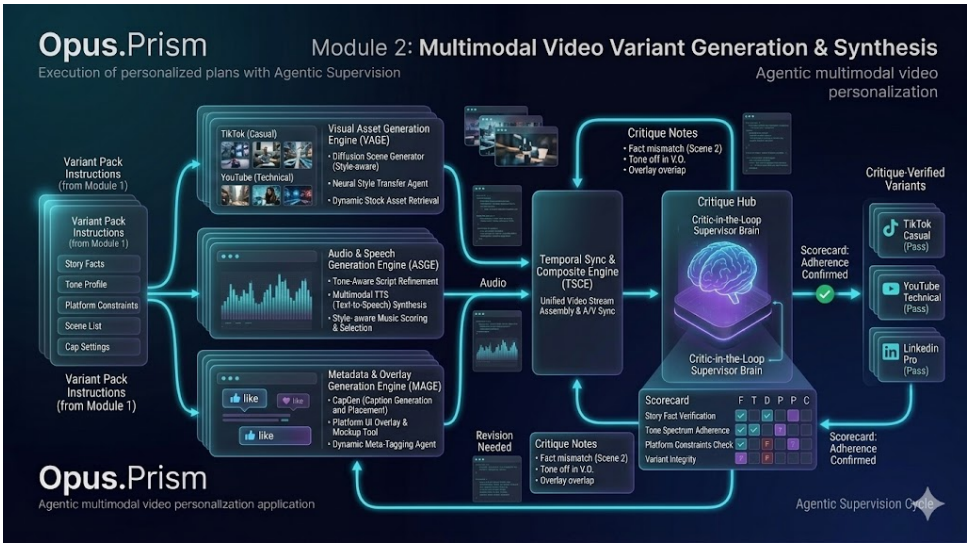

Figure 2: Generation of structured Variant Packs.

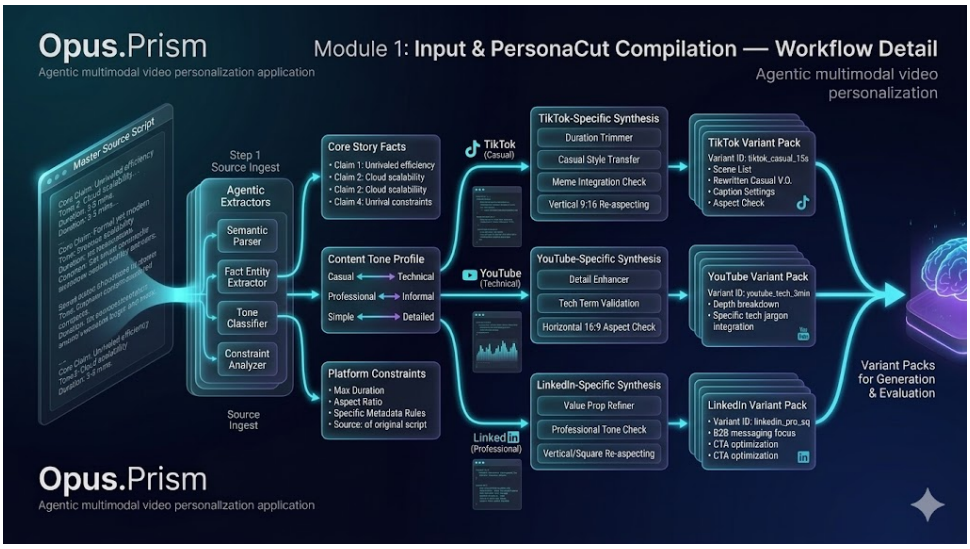

Figure 3: TextGuard QA identifying hallucinated text overlays.

3.3 TextGuard QA: The Second Critic

Once an MP4 is rendered via an external source based on the JSON Variant Pack, the user uploads it to the TextGuard QA tab (Figure 3). Generating text within video frames is notoriously unreliable; TextGuard acts as the visual unit testing suite.

Frame Extraction and OCR Matching The FastAPI backend utilizes `ffmpeg` to extract frames at 1 fps. Each frame is passed horizontally through an Optical Character Recognition (OCR) engine (`EasyOCR`). TextGuard then runs a `rapidfuzz` string similarity algorithm, comparing the visually detected text bounding boxes directly against the *Expected Captions* dictated by the original Agent 3 Writer.

VLM Inspection and Video Patching If the string similarity matcher identifies a severe mismatch (e.g., "Brwvv Perfctn" vs. "Brewed Perfection"), the isolated frame is handed off to `Qwen/Qwen2.5-VL-7B-Instruct`. The Vision-Language Model diagnoses

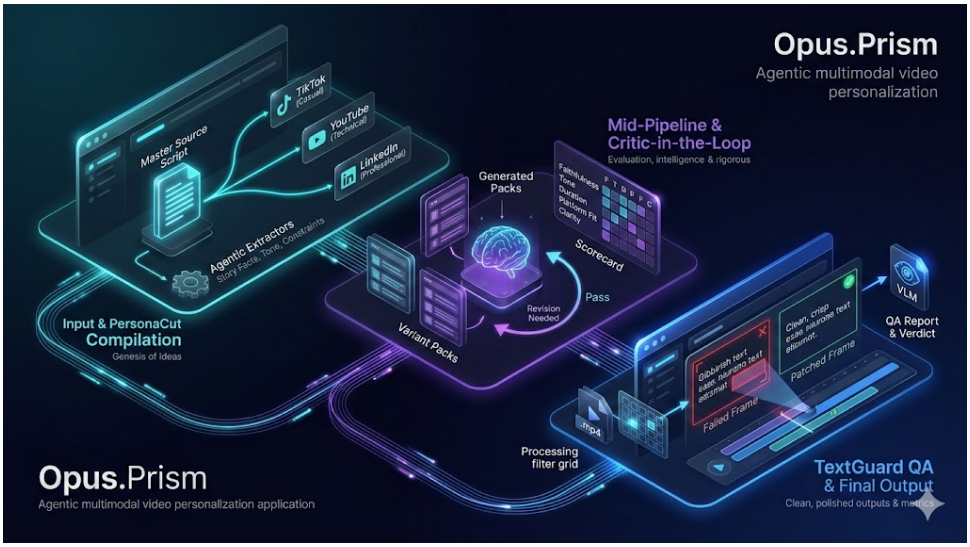

Figure 4: The VLM Verdict and the patched, corrected video output.

the failure, determining whether the text is genuinely hallucinated text, garbled by artifacts, or simply styled strangely.

If flagged as a failure, TextGuard dynamically redraws a clean SVG text box over the coordinates of the corrupted frame, utilizing `ffmpeg` filters to generate a final **Patched MP4** seamlessly bridging over the AI errors (Figure 4).

4 API Configuration & Integrations

Opus.Prism relies on highly configurable '.env' setups to seamlessly hook into local or hosted infrastructures:

```
# Hugging Face Open-Weight Models via HF Inference API
LLM_VARIANT_BASE_URL=https://router.huggingface.co/v1
LLM_VARIANT_MODEL_ID=Qwen/Qwen2.5-72B-Instruct

LLM_CHAT_BASE_URL=https://router.huggingface.co/v1
LLM_CHAT_MODEL_ID=Qwen/Qwen2.5-7B-Instruct

VLM_QA_BASE_URL=https://router.huggingface.co/v1
VLM_QA_MODEL_ID=Qwen/Qwen2.5-VL-7B-Instruct

# HF token for all model calls
OPENAI_COMPAT_API_KEY=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

# Orchestrator & Integrations
OCR_PROVIDER=easyocr
USE_CREWAI=false
ELEVENLABS_API_KEY=sk_bf72e7c681e6f25a...
```

5 Conclusion

Opus.Prism acts as both a specialized "compiler" for narrative personalization (PersonaCut) and a rigorous "unit testing suite" for generated video media (TextGuard). By bridging these two components with a pervasive critic-in-the-loop and utilizing state-of-the-art Qwen open-weight models, it automates the tedious workflows that video creators battle daily, generating personalized media safely and intelligently.